

Adaptive Spline Trajectory Planning with Simulated Annealing

1.0 Introduction

The goal of this project was to develop a custom path and motion planner to drive along curved [spline] paths. This project achieves this with a TurtleBot3, navigating a maze and crowded spaces in the ROS2-Gazebo Sim Environment. **Each spline path is trained to adapt and conform to its environment for robot safety when driving.**

2.0 Background

Our robot must navigate from a rest position to a target position. The environment is a PGM map, loaded from a Map Server node. The map is dilated and inflated to create a configuration space (c-space). The c-space is a discretized (vectorized) 2D graph, scaled to fit the continuous space the robot navigates. Each graph cell is either free or occupied by an obstacle or wall. Free spaces are places the robot can safely drive through.

Trajectory planning involves (1) path planning and (2) motion planning. The first is handled with heuristics and an A* graph search on the c-space between where the robot starts and its target position. The result is a connected chain of edges between free cells in the c-space. Common path planners, such as Point-to-Point (P2P), then parse a path from those free cells. However, common planners will follow the A* path closely to ensure robot safety. **This becomes a problem for spline planners.**

2.1 Project Motivation

Our robot creates paths to follow from quintic spline polynomials. Splines create organic and curved geometric paths. These are excellent for non-holonomic motion and smooth maneuvering. Our planner parses waypoints from an A* search to interpolate the spline path.

Splines take on several shapes and forms based on their curvature and the waypoints they interpolate.

The A* search provides potential waypoints. But waypoints must be selectively chosen. As seen in **Figure 1**, a raw path is created from points connected by a dashed line, where the spline interpolates a subset of those points.

This becomes a balancing act. In our case, the A* path is the dashed line. Fewer waypoints means the spline path is smoother and can have larger maneuvers. But, it may be unclear what shape that spline takes on, and if it risks colliding with an obstacle. Conversely, using more waypoints constrains the spline onto the A* path that will ensure it does not collide with obstacles.

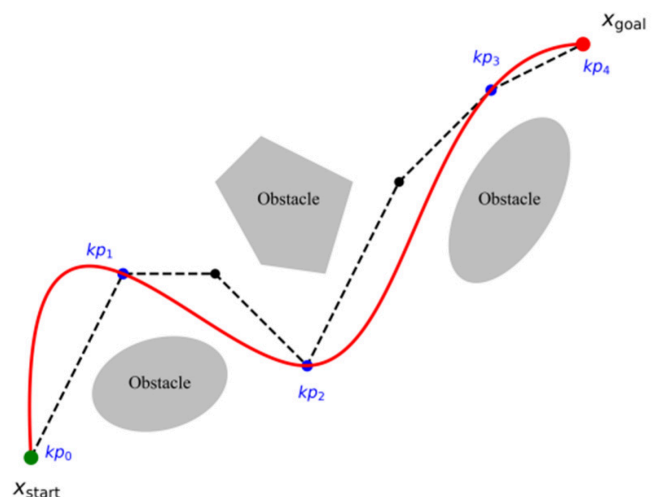


Figure 1: Illustrates spline path planning. [1]

2.2 Exploration-Exploitation Bottleneck on Splines and Safety

If too many waypoints are chosen from the A* path, then A* will have more influence over the path geometry than the curvature of the splines created. The balancing act from before becomes a trajectory bottleneck. The curvature and geometric malleability of splines is minimized when the number of waypoints and safety are maximized. **Figure 1** shows this trade-off, in which every other waypoint is rejected.

The spline path also follows the same homotopy as the original path. Viewing **Figure 1** from left to right, both the original and spline path travel over the first obstacle and weave under the second, and over the third in the same topological way. Although this allows for deterministic path planning, there are benefits to differing spline homotopy from the original path. For instance, consider if the spline in **Figure 1** could not pass through point **kp2**, and the spline instead traveled over atop the second obstacle from point **kp1** to **kp3**. **A spline path like that may turn out to be shorter or smoother.**

But giving splines the ability to explore different homotopies from the original path brings back the bottleneck of exploring at the expense of safety; this is especially true in environments that can change. **Therefore, adaptive spline planning that avoids obstacles could reconcile exploration beyond A* with safety.**

3.0 Motion Controls

Trajectory planning involves (1) path planning and (2) motion planning. This section will briefly cover the motion planning. Motion planning is based on robot odometry and the interpolated spline path it follows.

3.1 Trapezoidal Motion Profile

Once provided the spline path to follow, the navigation node (**nav_node**) computes linear speeds. Speed is calculated at each point along the discretized spline path. Linear speed is determined based on acceleration and maximum speed constraints.

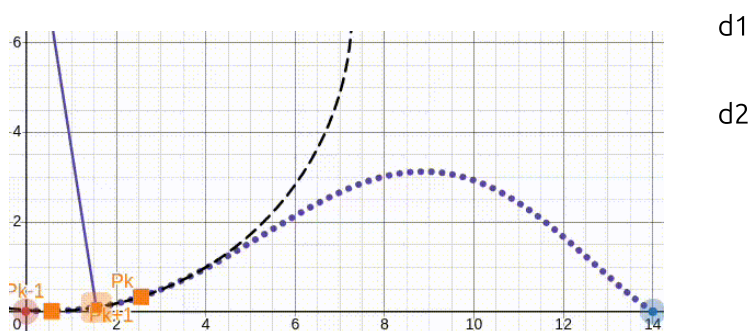


Figure 2: Illustrates approximating angular motion from spline path. [A custom demonstration]

3.2 PID Feedback with Stanley Controller

4.0 Adaptive Splines

4.1 Cost-Optimization Problem

4.2 Optimizing Splines with Simulated Annealing

4.3 Initial Tests

5.0 Trajectory Planning Package Pipeline

5.1 Waypoint Parsing

- A* -> Parsing -> Centroid Snapping = Waypoints

5.2 Waypoint Dropout

5.3 Final Heading Tuning

5.4 ROS2 Package Design and Pipeline

6.0 Results

7.0 Conclusion

8.0 References

[1] Sun, Z., Luo, Q., Zhang, Z., Peng, Y., Liu, Q., Zheng, S., & Liu, J. (2025). An Integrated Path Planning and Tracking Framework Based on Adaptive Heuristic JPS and B-Spline Optimization. *Machines*, 13(8), 710. <https://doi.org/10.3390/machines13080710>